

IA Aplicada com LLMs

Aula 07 — O que o embedding não vê

Antes de tudo: o par de controle

Um valor de cosseno isolado **não é interpretável**: não há escala para classificar 0,84 como alto ou baixo.

	Texto
A	"a despesa foi aprovada pelo analista"
B	"a previsão do tempo indica chuva no litoral norte"

Dois textos **sem relação alguma**. O valor obtido é o **piso prático da escala**.

Todas as medições seguintes se leem **em relação a esse piso**, e não em relação a zero.

Experimento 1

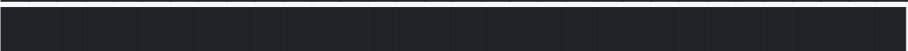
Negação

Negação — a previsão

	Texto
A	"a despesa foi aprovada pelo analista"
B	"a despesa foi reprovada pelo analista"

Uma palavra de diferença. Para um leitor humano, a distância entre as duas é **total**: uma libera o pagamento, a outra o impede. **A previsão da turma é registrada antes da medição.**

Negação — o resultado

NEGAÇÃO	0.9400		<- textos SEM RELAÇÃO
CONTROLE	0.6557		

É o momento de virada do encontro — o equivalente aos rótulos errados da Aula 03 e ao laço da Aula 05.

Para o vetor, "aprovado" e "reprovado" estão **mais próximos entre si** do que qualquer um deles está de uma frase sobre o tempo.

Negação — o conserto

A polaridade é um **campo**, não um trecho de texto.

```
# ERRADO – a polaridade viaja dentro do texto da consulta
indice.buscar("pareceres reprovados por falta de nota fiscal", k=5)

# CERTO – o campo filtra, o vetor decide só o assunto
indice.colecao.query(
    query_embeddings=[vetor("falta de nota fiscal")],
    where={"veredito": "reprovado"},    # ANTES de ordenar
    n_results=5,
)
```

Filtrar **antes** reduz o espaço de busca. Filtrar **depois** descarta bons resultados que já perderam lugar para os errados.

Experimento 2

Número

Número

	Texto
A	"o teto de reembolso é de R\$ 120,00 por refeição"
B	"o teto de reembolso é de R\$ 260,00 por refeição"

NÚMERO	0.9836	
CONTROLE	0.6557	

Uma despesa de R\$ 200,00 é **reprovada sob A e aprovada sob B**. Para o vetor, os dois textos são quase o mesmo.

A Aula 07 explora isso: o corpus contém a versão vigente e uma versão revogada do mesmo artigo.

Número — o conserto

O vetor **localiza** a regra. O código **aplica** a regra.

```
# ERRADO – pedir ao vetor que compare grandezas
indice.buscar("uma refeição de R$ 340,00 está dentro do teto?", k=3)

# CERTO
artigo = indice.buscar("teto de refeição em viagem internacional", k=1)[0]
dentro = despesa.valor <= artigo["valor"]      # 340.00 <= 260.00 -> False
```

Achar o **Art. 4º §2º** entre quarenta artigos é difícil, e o vetor faz bem.

`340 <= 260` é uma linha de Python — exata, testável, e sempre certa.

Experimento 3

Entidade

Entidade

consulta: "despesa D-4471"

D-4471 0.9615

D-4472 0.9601

D-4183 0.9588

D-9902 0.9574

Mil identificadores do mesmo formato ocupam posições quase idênticas.

O vetor não sabe distinguir identificadores — ele representa "isto é um código de despesa", não *qual* código.

Entidade — o conserto

Identificador bem formado **não é pergunta de busca**. É rota.

```
IDENTIFICADOR = re.compile(r"\b[DF]-\d{4}\b")           # D-4471, F-088

def recuperar(pergunta: str) -> list[dict]:
    if achado := IDENTIFICADOR.search(pergunta):
        return banco.por_id(achado.group())             # exato, zero chamadas
    return indice.buscar(pergunta, k=3)                 # só aqui o vetor entra
```

Seis linhas eliminam a cegueira inteira para essa classe de pergunta.

É o degrau **regex** da escada — o mais barato, e o primeiro a tentar.

Experimento 4

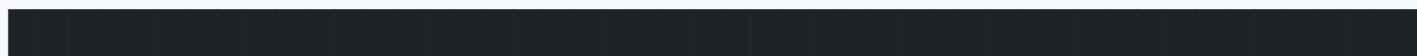
Tempo

Tempo

	Texto
A	"em março de 2026, o teto de refeição passou a ser R\$ 120,00"
B	"em agosto de 2026, o teto de refeição passou a ser R\$ 150,00"

TEMPO

0.9648



Os dois são **verdadeiros**. Apenas um está **vigente**.

O vetor não tem noção de anterioridade, e a busca traz os dois com peso equivalente.

Plantado aqui, colhido na Aula 08 — quando dois fatos verdadeiros em datas diferentes disputarem a mesma pergunta.

Tempo — o conserto

A data é **metadado**. O desempate é uma linha.

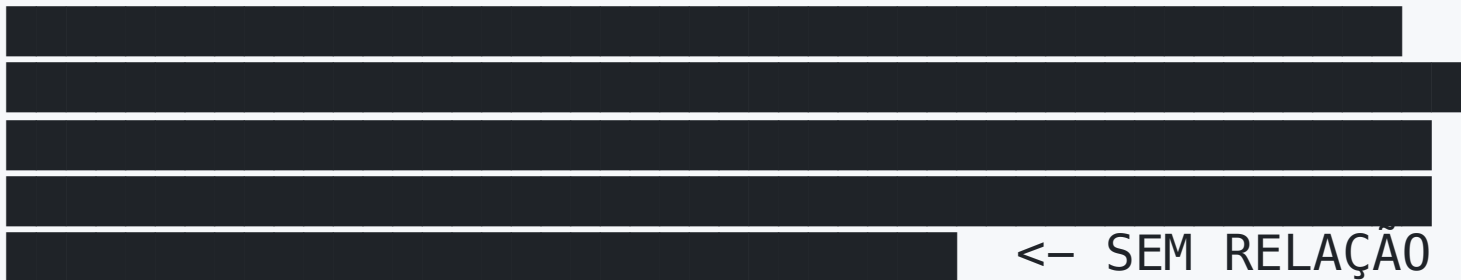
```
# ERRADO – delegar a vigência ao modelo, com os dois textos no contexto
gerar("Considere apenas a versão vigente.\n" + contexto)

# CERTO
def mais_recente(candidatos, campo="data"):
    return max(candidatos, key=lambda c: c[campo]) if candidatos else None

vigente = mais_recente(indice.buscar("teto de refeição", k=2))
```

O jeito errado **não é uma versão mais fraca do certo**: ele falha justamente quando as duas versões são parecidas — que é sempre.

O quadro completo, lido contra o controle

NEGAÇÃO	0.9400	
NÚMERO	0.9836	
ENTIDADE	0.9738	
TEMPO	0.9648	
CONTROLE	0.6557	<- SEM RELAÇÃO

Nenhuma das quatro se aproxima do piso.

Não se trata de o modelo distinguir mal. Trata-se de o modelo **não estar medindo aquilo**.

As quatro são a mesma propriedade

O embedding representa contexto de ocorrência. Não representa polaridade, magnitude, identidade nem ordem.

A comparação é sobre	O instrumento correto
polaridade de uma decisão	campo com domínio fechado, filtrado na consulta
magnitude de um valor	operador de comparação numérica
identidade de um registro	igualdade, ou índice invertido
anterioridade	carimbo de tempo e <code>max ()</code>
assunto	similaridade de vetores

A última linha é o que sobra — e é bastante

Localizar, entre quarenta artigos, os três que tratam do **assunto** da pergunta é um problema real, difícil por outros meios, e que a busca semântica resolve bem. **O erro não está em usar embeddings, e sim em usá-los para as outras quatro linhas.**

A conclusão prática

Os quatro consertos são **o mesmo conserto**, e nenhum deles envolve o modelo:

Cegueira	O conserto	Custo
Entidade	<code>regex</code> roteia para consulta direta	zero
Negação	<code>where</code> por campo de domínio fechado	zero
Tempo	carimbo no metadado, <code>max()</code> no código	zero
Número	o vetor localiza, o <code><=</code> decide	zero

Regra determinística onde ela existe. O trabalho não é escolher um modelo melhor — é **tirar do vetor** a decisão que ele não sabe tomar.

É a mesma conclusão do roteador da Aula 05 e da tática zero de lá (nota 03, §4), aplicada agora à recuperação: **o caminho mais barato costuma ser o que não envolve o modelo**.

A máscara — e o empate que ela revela

Se `<ID>`, `<VALOR>` e `<DATA>` são ruído para o vetor, por que embuti-los?

```
MES = "janeiro|fevereiro|março|...|novembro|dezembro"

MASCARAS = [
    (re.compile(r"\b[A-Z]-\d{3,4}\b"), "<ID>"),
    (re.compile(r"R\$ ?[\d.]+\d{2}"), "<VALOR>"),
    (re.compile(rf"\b(?:{MES}) de(?:19|20)\d{{2}}\b", re.I), "<DATA>"),
]

def canonicalizar(texto: str) -> str:
    for padrao, marca in MASCARAS:
        texto = padrao.sub(marca, texto)
    return texto
```

Aplicada aos pares dos experimentos **2, 3 e 4**, ela produz textos **idênticos**.

O cosseno vai a 1,0000 — e isso é o ganho

ENTIDADE	-> "análise da despesa <ID> do funcionário <ID>"	0.9738	-> 1.0
NÚMERO	-> "o teto de reembolso é de <VALOR> por refeição"	0.9836	-> 1.0
TEMPO	-> "em <DATA>, o teto de refeição passou a ser <VALOR>"	0.9648	-> 1.0

Não é medição, é aritmética: textos idênticos dão o mesmo vetor.

A máscara **não conserta** a cegueira. Ela a torna **visível**.

Os 26 milésimos que separavam **D-4471** de **D-4472** eram **ruído puro** — e o sistema os usava para ordenar a lista que o aluno lia como resultado.

Com o empate explícito, ordenar por similaridade fica obviamente impossível.
Sobra o caminho certo: **filtrar por metadado**.

A armadilha: duas coleções de texto, não uma

Se o texto mascarado for o texto indexado, o modelo recebe

"o teto é de `<VALOR>` por refeição" — e não tem como responder.

```
colecacao.add(  
    documents=[c["texto"] for c in chunks],           # ORIGINAL -> ao LLM  
    embeddings=gerar_matrix_embeddings(  
        [canonicalizar(c["texto"]) for c in chunks]).tolist(), # CANÔNICO -> compara  
    metadatas=[...],  
)
```

O `IndiceChroma` já separa os dois — ele passa os embeddings prontos. **Uma linha de diferença.**

E a consulta passa pela **mesma** função: canonicalizar só o corpus é pior do que não canonicalizar nada.

Três condições, e uma fragilidade

Extraír, depois mascarar	o valor tem de estar no metadado antes de sair do texto
Só o que é ruído	<i>"o que mudou na política de 2024?"</i> — aqui o ano é o assunto
A máscara entra no carimbo	dois índices com máscaras diferentes não são comparáveis

A fragilidade está no próprio código. É `[A-Z]-\d{3,4}` e não `[DF]-\d{4}`, porque o corpus tem `D-4471` (quatro dígitos) e `F-088` (três). Uma máscara que pega um id e deixa o outro produz textos **parcialmente** iguais — com um resíduo arbitrário decidindo a ordem.

A regex custa zero e quebra quando a redação varia. Aqui, **em silêncio**.

Nenhuma das quatro é curiosidade

Cegueira	Reaparece em	Como
Negação	Aula 06, nota 03 · nota 04 desta aula · Aula 08	"chegou" e "não chegou" na mesma rota
Número	Aula 07, falha 4	duas versões do artigo, só o teto muda
Tempo	Aula 07 · Aula 08	qual das duas é a vigente
Entidade	Aula 10, nota 03	ids de ferramentas MCP se confundem

A **negação** já cobrou o preço dela antes do RAG: o router da Aula 06 errou a rota **sem nenhum modelo escrever uma palavra**.

O que fica

- O piso da escala não é zero — **meça contra um par de controle**
- Negação, número, entidade e tempo: **quatro ângulos da mesma propriedade**
- O vetor mede **assunto**, e só
- Regra determinística onde ela existe
- **Mascarar** o que o vetor não usa não conserta a cegueira — **revela** o empate
- As cegueiras são problema de **representação**, não de geração

E por que isto está aqui, e não na Aula 06

Porque agora tem consequência visível.

na aula 06	o erro aparece como número na tela
aqui	o erro vem embrulhado numa resposta bem redigida, com fonte citada

O RAG não cria a falha: torna-a plausível. O trecho recuperado é o mesmo nos dois casos; o que muda é a forma com que ele chega ao leitor.